Project Check-In Preprint

Exploration & Acceleration: How Generative AI Reshapes Development Practices

Nikhil Anand

University of Massachusetts, Amherst

nikhilanand@umass.edu

Abstract

Generative AI tools such as GitHub Copilot and ChatGPT are increasingly integrated into software development workflows, yet their influence on developers' problem solving strategies remains insufficiently understood. This paper presents a literature survey of ten recent studies examining how generative AI affects developer behavior during programming tasks. We adopt a hybrid analytical approach that first defines a taxonomy of AI-assisted interactions and then analyzes their impact across key stages of problem-solving, including problem understanding, solution generation, implementation, and debugging. Our synthesis shows that generative AI shifts developers from exploration-driven problem-solving toward solution selection and refinement, reducing cognitive effort while introducing risks of over-reliance and shallow reasoning. The impact varies across stages: conversational tools primarily support problem understanding, while suggestion-based and generative tools accelerate implementation and debugging. We further identify cross-cutting patterns in trust, prompting behavior, and learning. Finally, we highlight gaps in existing work, including limited longitudinal evidence and insufficient analysis of expertise-dependent effects. These findings provide a structured perspective on how generative AI reshapes developer problem-solving and suggest directions for future research.

CCS Concepts • **Software and its engineering** → **Software development process management**; • **Computing methodologies** → *Multi-agent systems*; Information extraction; Natural language generation

1. Introduction

Generative AI tools powered by large language models (LLMs), such as GitHub Copilot and ChatGPT, are rapidly becoming integral to software development workflows. These tools provide a wide range of capabilities, including code completion, natural language driven code generation, explanation of programming concepts, and automated debugging assistance. Unlike traditional code recommendation systems, Generative AI enables interactive and iterative engagement through conversational interfaces, allowing developers to specify intent, refine outputs, and explore alternative solutions in real time [6, 33]. Recent empirical studies report widespread adoption of such tools and demonstrate measurable improvements in task completion speed and perceived productivity [7, 29]. As a result, Generative AI is reshaping how developers interact with code and development environments.

Beyond productivity improvements, emerging research has begun to characterize how Generative AI influences developer behavior during programming tasks. Studies have observed changes in code writing practices, debugging strategies, and patterns of interaction with development tools [6, 22]. Developers increasingly engage in prompt based workflows, iteratively refining natural language queries to obtain desired outputs, and often integrate AI-generated code directly into their programs with varying levels of verification [20, 33]. These interactions introduce new dynamics in programming, including shifts in how solutions are explored, how alternatives are evaluated, and how correctness is established. At the same time, prior work highlights variability in usage patterns across developers, tasks, and tool modalities, reflecting differences in expertise, trust, and familiarity with AI-assisted programming environments [20, 22].

To provide a structured understanding of these developments, this paper presents a literature survey of ten recent studies examining developer interactions with Generative AI tools. We adopt a hybrid analytical approach that first defines a taxonomy of AI-assisted programming interactions, capturing distinct modes of engagement such as suggestion-based, conversational, and generative workflows. Building on this taxonomy, we analyze how these interaction types influence different stages of the software development process, including problem understanding, solution genera-

tion, implementation, and debugging. By synthesizing findings across diverse empirical settings and study designs, this work offers a consolidated view of how Generative AI tools shape developer behavior and interaction patterns in programming tasks.

1.1 Motivation

The dream of an "AI assistant" working alongside the programmer has captured our imagination for several decades now, giving rise to a rich body of work from both the programming languages [13, 24, 31] and the machine learning [36] communities. Thanks to recent breakthroughs in large language models (LLMs) [21, 34] this dream finally seems within reach. OpenAI's Codex model [10, 28], which contains 12 billion model parameters and is trained on 54 million software repositories on GitHub [14], is able to correctly solve 30–70% of novel Python problems, while DeepMind's AlphaCode [15, 21] ranked in the top 54.3% among 5000 human programmers on the competitive programming platform Codeforces. With this impressive performance, large code generating models are quickly escaping research labs to power industrial programming assistant tools, such as Github Copilot.

Generative AI has emerged as a transformative paradigm across a wide range of domains, enabling systems to produce human-like text, code, and other artifacts with increasing fidelity. Advances in large language models (LLMs) have demonstrated strong capabilities in reasoning, language understanding, and generation, leading to their rapid adoption in real-world applications [8, 9]. These models have shifted the role of AI from purely analytical systems to interactive assistants capable of supporting complex cognitive tasks. As a result, Generative AI is increasingly positioned as a collaborative partner in knowledge work, fundamentally altering how users interact with computational systems and solve problems.

In the context of software development, Generative AI tools such as GitHub Copilot and ChatGPT provide direct support for programming activities, including code generation, explanation, and debugging. Empirical studies have shown that these tools can improve developer productivity, reduce task completion time, and assist in learning unfamiliar concepts [20, 29, 33]. Developers are able to offload routine implementation effort and leverage AI-generated suggestions to accelerate development workflows. At the same time, these tools introduce new interaction patterns, such as prompt-based programming and iterative refinement, which reshape how developers approach coding tasks. This evolution raises important questions about how developers engage with problems, evaluate solutions, and integrate AI assistance into their workflows.

At an industry level, the integration of Generative AI into development environments has significant implications for productivity, collaboration, and software engineering practices. Organizations are increasingly adopting AI-assisted programming tools to improve efficiency and reduce development costs, while also redefining skill requirements for developers [7, 23]. Despite these advancements, there remains limited understanding of how Generative AI influences the underlying problem-solving strategies of developers across different stages of programming. Existing studies often focus on performance metrics or usability, leaving a gap in understanding the cognitive and behavioral shifts induced by AI assistance. This highlights the need for a structured synthesis of current research to better understand how Generative AI reshapes developer problem-solving and to inform the design of future tools and practices.

2. AI-Assisted Programming

To systematically analyze how generative AI influences developer problem-solving, we first characterize the different modes through which developers interact with AI systems during programming tasks. Rather than categorizing tools based on underlying models or capabilities, we focus on interaction patterns that reflect how developers engage with AI assistance in practice. In particular, we distinguish between two primary modes of interaction: exploration and acceleration. Exploration-oriented interactions involve using AI to understand problems, generate ideas, and examine alternative solutions, often through iterative and conversational engagement. In contrast, acceleration-oriented interactions focus on efficiently completing tasks by leveraging AI-generated suggestions or code with minimal deliberation. This distinction provides a useful lens for analyzing how AI support shapes developer behavior across different stages of problem-solving.

Generative AI as an Autocomplete Tool When solving structured programming tasks such as those found in Advent of Code [35], developers often decompose the problem into smaller subtasks, such as parsing input into appropriate data structures. Even when the solution approach is clear, generative AI tools can be used to accelerate implementation by assisting with code completion and reducing the effort of recalling syntax or API details. Developers may provide contextual comments to guide the tool, after which inline suggestions appear during pauses in typing. These suggestions, including end-of-line completions (Figure 1), enable faster code writing while maintaining the developer's original intent. In this case (Figure 1), Copilot suggests the correct API function invocation to split the rule into its left- and right-hand sides. To come up with this suggestion, Copilot relies on the context, i.e. some amount of source file content preceding the cursor, which can include both code and natural language comments, as is the case in our example.

Generative AI as an Exploration Tool When developers encounter unfamiliar tasks, generative AI can be used as an exploration tool to understand both the problem and potential solution approaches. Developers may begin by express-

```
# rules are formatted like:
# AB => C
def parse_input(filename):
    with open(filename) as f:
        template, rules = f.read().split("\n\n")
        for rule in rules:
            rule_parts = rule.split(" => ")
```

Figure 1: An example of an end-of-line suggestion from Copilot, which completes the current line of code based on the context provided by preceding code and comments.

```
You, now | 1 author (You)
1  import matplotlib
2  import matplotlib.pyplot as plt
3
4  def read_first_digits_from_file(filename):
5      with open(filename) as file:
6          data = file.read().splitlines()
7      return [int(line[0]) for line in data]
8
9  fib_first_digits = read_first_digits_from_file("fib.
10 inverse_first_digits = read_first_digits_from_file("
11
12 # Plot the first digits of the Fibonacci
13 # sequence as a histogram      You, now • Uncommitte
14
15
16
17
```

Figure 2: Example code where a user needs a more lengthier suggestion

ing the task through natural language prompts (lines 12-13 of Figure 2), which enables the system to generate multiple candidate solutions. By invoking the multi-suggestion interface (Figure 3), they can examine several alternatives, identify common patterns, and observe relevant API usage, helping build confidence in the generated outputs. This process allows developers to select suitable code snippets while also gaining an understanding of underlying abstractions and libraries. To ensure correctness, the generated code is typically executed and inspected, serving as a form of validation. Such interactions are iterative and deliberate, involving prompt refinement, comparison of alternatives, and verification, reflecting an exploratory mode of problem-solving.

3. Taxonomy of AI-Assisted Programming

3.1 Suggestion-Based Interactions

Suggestion-based assistance represents a class of AI interactions where developers receive real-time code completions based on the current programming context. These interactions are typically lightweight and minimally intrusive, requiring little to no explicit input beyond the code being written. By providing inline or on-demand suggestions, such systems aim to accelerate implementation by reducing

```
3  =======
4
   Accept Solution
5  # Plot the first digits of the Fibonacci sequence as
6  plt.hist(fib_first_digits, bins=range(0, 10))
7  plt.title("Fibonacci sequence")
8  plt.xlabel("First digit")
9  plt.ylabel("Number of occurrences")
10 plt.savefig("fib.png")
11
12 =======
13
   Accept Solution
14 # Plot the first digits of the Fibonacci sequence as
15 plt.hist(fib_first_digits, bins=range(0, 10))
16 plt.title("Fibonacci sequence")
17 plt.xlabel("First digit")
18 plt.ylabel("Number of occurrences")
19 plt.show()
20
21 =======
22
   Accept Solution
23 # Plot the first digits of the Fibonacci sequence as
24 plt.hist(fib_first_digits, bins=10, range=(0, 10))
25 plt.title("Fibonacci sequence")
26 plt.xlabel("First digit")
27 plt.ylabel("Number of occurrences")
28 plt.savefig("fib.png")
29
```

Figure 3: An example of Generative AI being used as an exploration tool, where the developer provides a natural language prompt to generate multiple candidate solutions.

keystrokes, aiding recall of syntax and APIs, and streamlining routine coding tasks. In this section, we categorize suggestion-based assistance into distinct subtypes based on the scope and presentation of generated code.

3.1.1 End-of-Line Completion

End-of-line completion represents the most granular form of suggestion-based assistance, where the AI system predicts and completes the remainder of a partially written line of code. These suggestions are typically short, context-aware, and require minimal cognitive overhead to evaluate, enabling developers to maintain flow during implementation. As illustrated in Figure 1, such completions often rely on local context and common coding patterns to provide syntactically correct continuations. Prior work shows that developers frequently accept these suggestions due to their low cost of verification and seamless integration into the coding process [6]. This interaction mode primarily reduces keystrokes and supports rapid code construction without significantly altering higher-level problem-solving strategies.

3.1.2 Multi-Line / Block Completion

Multi-line or block completion extends suggestion-based assistance by generating larger code segments, such as entire functions, loops, or conditional structures, based on the

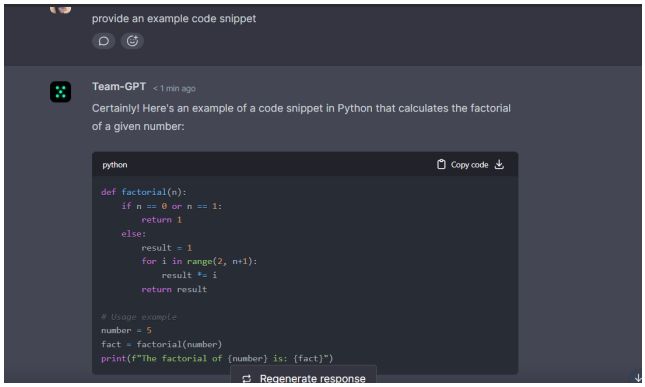

Figure 4: An example of conversational assistance, where the developer engages in a natural language dialogue with the AI to explore the problem and refine solutions.

surrounding context and developer intent. These suggestions provide more substantial acceleration benefits by automating repetitive or boilerplate code generation, allowing developers to focus on higher-level logic. However, larger completions require greater cognitive effort to evaluate and verify, as they may introduce subtle errors or assumptions [20, 33]. Studies indicate that while developers benefit from increased productivity, they may also rely on these suggestions without fully understanding the generated code, highlighting a trade-off between efficiency and comprehension [20].

3.1.3 Context-Aware Pattern Completion

Context-aware pattern completion refers to suggestions that capture recurring coding patterns or idioms across files or projects, enabling the AI system to generate code that aligns with broader structural or stylistic conventions. These suggestions go beyond local syntax completion by incorporating higher-level contextual signals, such as variable usage, naming conventions, or previously implemented logic. As a result, they can assist in maintaining consistency and reducing redundancy in codebases [31]. However, reliance on such patterns may reinforce existing coding habits and limit exploration of alternative implementations, subtly influencing developer decision-making during implementation.

3.2 Conversational Assistance

Conversational assistance refers to interaction modes in which developers engage with generative AI through natural language dialogue (Figure 4) to explore problems, request explanations, or refine solutions [20, 33]. Unlike suggestion-based systems, these interactions are explicit and iterative, allowing developers to externalize their reasoning and guide the AI through prompts and follow-up queries. Prior work shows that such conversational workflows enable flexible problem exploration and support tasks such as understanding unfamiliar code, learning APIs, and clarifying requirements.

Conversational assistance can be broadly categorized into query-driven interaction and iterative refinement.

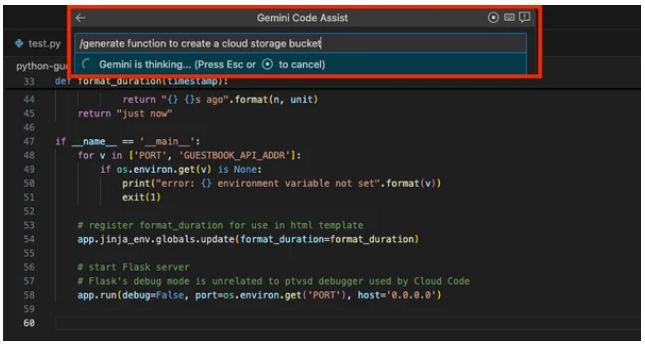

Figure 5: An example of generative assistance, where the developer provides a high-level intent and receives a substantial code output that implements the desired functionality.

1. In **query-driven interaction**, developers pose direct questions (e.g., requesting explanations or examples), using the AI as an on demand knowledge source.
2. In **iterative refinement**, developers progressively adjust prompts based on previous responses, engaging in a back-and-forth dialogue to converge on a desired solution.

These subcategories support exploration-oriented[1] workflows by enabling developers to decompose problems, compare alternatives, and refine their understanding, although the quality of outcomes depends heavily on prompt formulation and interpretation [6].

3.3 Generative / Transformational Assistance

Generative or transformational assistance refers to interaction modes where developers provide high-level intent and receive substantial code outputs, such as functions, program fragments, or refactored implementations. These interactions abstract away low-level details, allowing developers to focus on specifying desired functionality while the AI produces corresponding code artifacts (Figure 5). Such capabilities are central to modern LLM-based tools and are widely used to accelerate implementation and reduce manual coding effort [20, 29].

This form of assistance can be divided into code generation and code transformation. Code generation involves producing new code from natural language descriptions or partial context, while code transformation focuses on modifying existing code, such as refactoring, optimizing, or fixing bugs. Both subcategories provide significant efficiency gains but require careful validation, as generated outputs may introduce incorrect assumptions or incomplete logic. Studies indicate that developers often rely on these outputs for rapid progress, sometimes at the cost of reduced engagement with underlying problem details [6, 33].

[1] Exploration-oriented workflows involve using AI to iteratively explore, compare, and understand multiple solution possibilities before implementation.

3.4 Retrieval and Explanation Assistance

Retrieval and explanation assistance encompasses interaction modes where generative AI is used to access, summarize, or explain information relevant to programming tasks. Rather than generating new code directly, these interactions focus on supporting comprehension and decision-making by providing contextual knowledge, such as documentation summaries, error explanations, or conceptual clarifications [30]. This mode aligns closely with traditional information retrieval tasks but benefits from the contextual reasoning capabilities of LLMs [26].

This category can be divided into information retrieval and code explanation.

1. **Information retrieval** involves locating and summarizing relevant documentation or examples, reducing the need for manual search.
2. **Code explanation** focuses on interpreting existing code, errors, or outputs to help developers understand behavior and identify issues.

These subcategories are particularly valuable during debugging and learning phases, as they reduce cognitive overhead associated with searching and interpreting external resources. However, reliance on AI-generated explanations may affect how developers internalize knowledge and reason about code correctness [20].

4. Connection of these terms with Software Development Process

The adoption of generative AI tools in software engineering has accelerated significantly, with developers increasingly incorporating systems such as code generation assistants and conversational agents into their workflows. Large-scale studies and surveys indicate that AI-assisted programming tools are now commonly used for tasks ranging from code synthesis to debugging and learning [21, 27]. These tools are no longer peripheral utilities but are becoming embedded components of modern development environments, influencing how developers structure and execute programming tasks.

The interaction modes introduced by generative AI systems, as captured by the taxonomy presented earlier, exhibit distinct relationships with different stages of the software development process. Prior work has highlighted that developer-AI interaction is not uniform, but varies based on task context, intent, and level of abstraction [11, 25]. By mapping suggestion-based, conversational, generative, and retrieval-oriented assistance to development stages, we can better understand how these tools shape problem-solving strategies and workflow dynamics.

Conversational assistance is most closely associated with the ideation stage, where developers engage in problem understanding, requirement clarification[2], and exploration of potential solutions. Natural language interaction enables developers to iteratively refine their understanding and query the system for explanations, examples, and alternative approaches. Such interactions support exploratory reasoning and have been shown to facilitate learning and problem decomposition, particularly in unfamiliar domains [11, 16].

Suggestion-based and generative assistance primarily influence the development stage, where ideas are translated into executable code. Suggestion-based tools provide fine-grained completions that reduce the effort of writing syntactically correct code, while generative systems can produce larger functional units or transformations from high-level intent. These capabilities significantly accelerate implementation, allowing developers to focus on higher-level design decisions, although they may also introduce challenges related to verification and understanding [21, 27].

Retrieval and explanation assistance play an important role in the review stage, supporting debugging, validation, and reasoning about program behavior. By summarizing documentation, explaining errors, and interpreting code, these tools reduce the cognitive burden associated with information retrieval and comprehension. This enables developers to more efficiently identify issues and validate correctness, particularly in complex or unfamiliar codebases [25].

Finally, conversational and generative assistance extend into the documentation stage, where developers generate or refine descriptions of code, usage instructions, and comments. These tools can synthesize documentation directly from code or assist in rephrasing and structuring explanations through iterative interaction. As a result, they help streamline documentation practices [16] and improve consistency, further integrating AI support across the development lifecycle.

In the following section, we will examine each of these development stages in detail, providing a deeper analysis of how generative AI influences developer behavior and problem-solving strategies within each stage.

5. Impact of GenAI in the Software Development Process

The Software Development Life Cycle (SDLC) (Figure 6) is a structured process that guides software development through stages such as requirements elicitation, design, implementation, testing, and deployment. Among these, requirements elicitation is particularly critical, as it involves understanding stakeholder needs and translating them into formal specifications that drive all subsequent stages. This process is inherently human centric and relies heavily on communication, domain expertise, and iterative clarification. However, traditional approaches to requirements engineering often struggle with ambiguity, incomplete information,

[2] Here, we refer to the individual developer solving a problem, excluding Software Development Models, eg, Waterfall, Agile, etc.

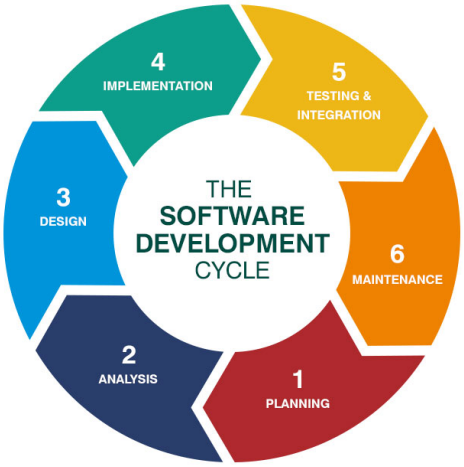

Figure 6: The Software Development Life Cycle (SDLC)

Table 1: Software development stages considered in this study

Stage	Small description
Requirements Elicitation	Gathering, clarifying, and structuring user needs and system expectations; involves understanding ambiguity, stakeholder communication, and defining what needs to be built.
Coding	Translating requirements into executable code; includes implementation, API usage, logic construction, and integration of components.
Debugging	Identifying, analyzing, and fixing errors or unintended behavior in code; involves reasoning about program execution and validating correctness.
Documentation	Creating explanations, comments, and supporting material for code and systems; helps with knowledge transfer, maintainability, and onboarding.

and the difficulty of training individuals in elicitation skills. In educational settings, this challenge is further compounded by the reliance on static artifacts such as interview transcripts, which allow learners to practice requirement extraction but fail to develop the interactive and exploratory skills required for real-world elicitation scenarios [2, 4].

The rise of Generative AI, particularly Large Language Models (LLMs), is beginning to transform how SDLC activities—especially requirements engineering—are performed. LLMs like ChatGPT have demonstrated strong capabilities in natural language generation and understanding, enabling them to assist in generating and refining requirements artifacts with high levels of abstraction, consistency, and understandability [3]. Empirical evidence suggests that AI-generated requirements can match or even outperform human-generated ones across several quality attributes, though challenges such as ambiguity and feasibility remain [5]. Beyond generation, LLMs can also simulate stakeholders in interactive settings, providing scalable and engaging environments for practicing elicitation skills and improving communication abilities [12]. While these advancements highlight the potential of Generative AI to augment human roles and improve efficiency across the SDLC, they also underscore the continued need for human oversight to ensure correctness, feasibility, and domain alignment.

For the purposes of this paper, we focus on four key stages of the SDLC that are particularly impacted by Generative AI: requirements elicitation, coding, debugging, and documentation (Table 1). These stages represent critical points in the development process where human-AI interaction can significantly influence developer behavior, problem-solving strategies, and overall productivity.

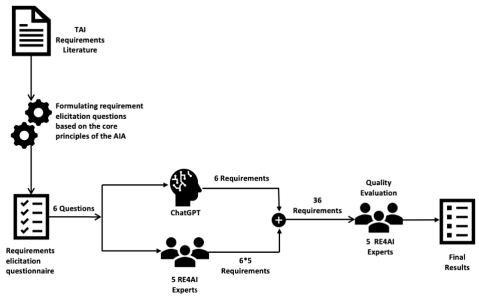

Figure 7: Two-stage process employed by Ronanki et al. [32]

5.1 Requirements Elicitation

5.1.1 Generating Synthetic Requirements via Trustworthy AI Principles

Ronanki et al. [32] employ a two-stage methodology: first, using ChatGPT to generate synthetic requirements for Trustworthy AI systems, and second, conducting interview-based surveys. Six elicitation questions were designed and posed both to ChatGPT (with a context prompt) and to five RE4AI experts, resulting in 36 responses (6 AI-generated, 30 expert-generated). These responses were then evaluated by another set of five RE experts across seven quality attributes: abstraction, atomicity, consistency, correctness, unambiguity, understandability, and feasibility. The principles of Trustworthy AI are grounded in prior work such as Kaur et al. [19].

The results (Figure 8) show that ChatGPT-generated requirements perform strongly across most quality attributes

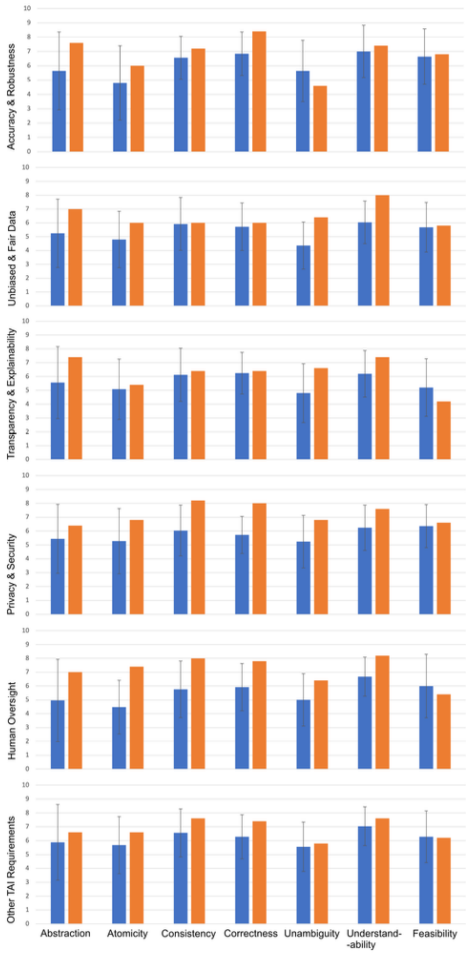

Figure 8: Comparative analysis of requirements quality attributes for ChatGPT-generated and human-generated requirements by Ronanki et al. [32]

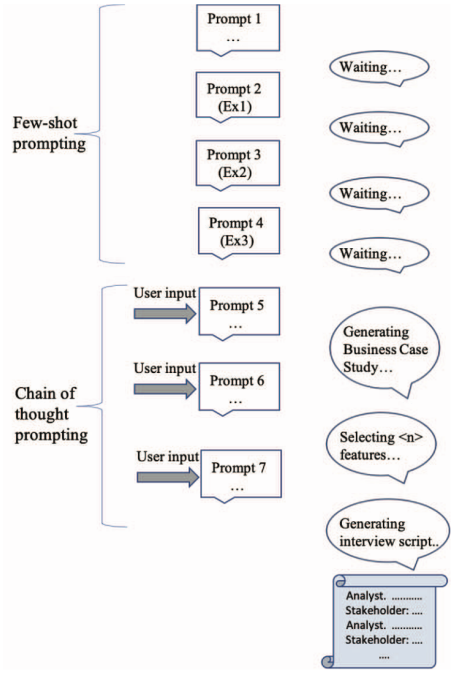

Figure 9: Batch generation approach for structured elicitation artifacts by Görer and Aydemir [17]

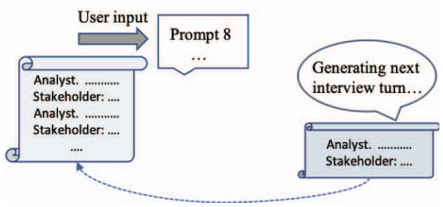

Figure 10: Iterative generation approach for structured elicitation artifacts by Görer and Aydemir [17]

and often outperform human-generated ones in terms of structure and clarity. However, limitations are observed in unambiguity and feasibility, with AI outputs sometimes being vague or impractical. Both AI and human responses occasionally miss critical aspects like safety and regulatory concerns, indicating the continued need for domain expertise. Overall, ChatGPT is effective as an assistive tool but requires human oversight for ensuring completeness and real-world applicability.

5.1.2 Structured Interview Scripts Generation via LLMs

Görer and Aydemir [17] propose a prompt engineering–based approach to generate structured elicitation artifacts using large language models. Their methodology combines few-shot and chain-of-thought prompting to guide models in producing business domain descriptions, feature sets, and multi-turn conversational artifacts. To support this, the authors introduce a graph-based representation of interactions, which is further decomposed into linear samples to improve model comprehension. Two generation strategies are explored: batch generation (Figure 9), where the entire artifact is produced at once, and iterative generation (Figure 10), where content is constructed incrementally with finer control over each step and injected variations.

The study finds that careful prompt design and task decomposition significantly improve the coherence and completeness of generated outputs. While models such as ChatGPT perform better in producing structured and consistent artifacts in batch settings, iterative approaches enable more precise control and explanation of specific interaction patterns. However, limitations remain in handling complex, non-linear conversational structures and certain nuanced scenarios. Overall, the results suggest that LLMs can effectively support the creation of structured elicitation material,

but require guided prompting and human oversight to ensure accuracy and contextual relevance. The generated scripts samples are available in their repository [18].

5.2 Development

5.2.1 Code Generation From Natural Language

Agarwal et al. [1] evaluated how LLMs translate natural language intent into executable code within an IDE-centric workflow. Their methodology involved selecting real-world methods from GitHub repositories, providing the model with the function signature, docstring, and surrounding file context, and prompting it to generate the method body. The generated code is then reintegrated into the original project and evaluated using syntax correctness and test pass rate, where the entire test suite is executed to verify functional behavior. This ensures that evaluation reflects realistic development conditions, where generated code must interact correctly with existing components rather than operate in isolation. The findings show that LLMs are generally effective at producing syntactically valid code and can often satisfy functional requirements when sufficient context and test coverage are provided. However, several limitations emerge: generated code may fail hidden edge cases not covered by tests, may rely heavily on prompt quality and context completeness, and can struggle with complex dependencies across files. Additionally, correctness is not guaranteed even when code appears plausible, emphasizing the gap between surface-level generation and true semantic understanding. These results highlight that while LLMs significantly accelerate initial implementation, their outputs require systematic validation and integration checks to ensure reliability in real-world development scenarios.

Language	Model	Syntax Correctness	Bugs Fixed
Python	GPT-4	96%	74%
	GPT-3.5	93%	68%
	CodeLlama	88%	39%
Javascript	GPT-4	92%	81%
	GPT-3.5	85%	74%
	CodeLlama	39%	26%
Typescript	GPT-4	83%	75%
	GPT-3.5	74%	75%
	CodeLlama	70%	30%
C#	GPT-4	98%	58%
	GPT-3.5	96%	65%
	CodeLlama	84%	50%

Table 2: Agarwal et al. [1] evaluated the effectiveness of LLMs in generating syntactically correct code and fixing bugs across multiple programming languages. The results show that while LLMs can produce high rates of syntactically correct code, their ability to fix bugs varies significantly

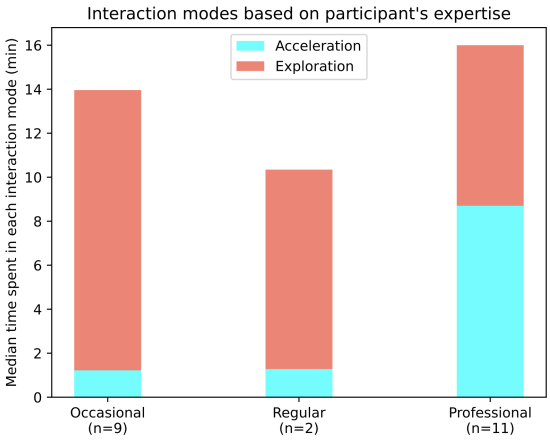

Figure 11: Time spent in each mode (acceleration vs exploration) by developers of different expertise levels, as reported by Barke et al. [6].

5.2.2 How are Developers interacting with LLMs?

A shift in developer behavior This study of Copilot by Bird et al. [7] shows that developers spend more time reviewing code (as suggested from Copilot or similar tools) than writing code. As AI-powered tools are integrated into more software development tasks, developer roles will shift so more time is spent assessing suggestions related to the task than doing the task itself (for example, instead of fixing a bug directly, a developer will assess recommendations of bug fixes). An underlying assumption is that this way of working—looking over recommendations from tools is more efficient than doing the task directly without help.

The Modes of Interaction Barke et al. [6], when finding a Grounded Theory for Copilot, finds that interaction with AI coding assistants like Copilot is fundamentally bimodal, consisting of an *acceleration mode*, where developers use the tool as an intelligent autocomplete to quickly implement known solutions, and an *exploration mode*, where they rely on it to discover approaches, APIs, or solution structures when uncertain. Developers transition between these modes based on task familiarity, spending significantly more time in exploration due to its higher cognitive demand (Figure 11). In acceleration, they prefer short, precise suggestions and validate them through quick pattern recognition, while in exploration they engage in deliberate behaviors such as writing natural language prompts, examining multiple suggestions, and validating outputs through execution, documentation, or static analysis. The study also highlights a shift in developer roles from code authorship to code evaluation and refinement, as developers increasingly review, edit, and adapt

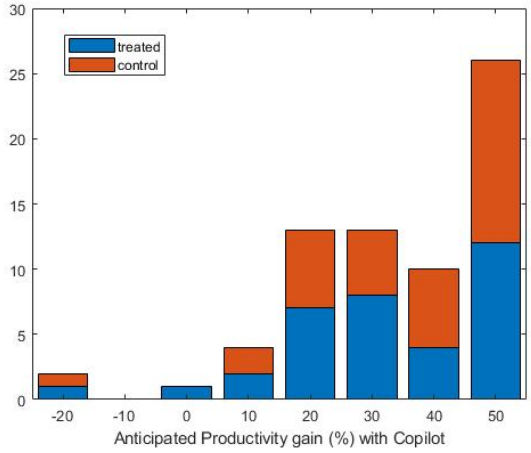

Figure 12: Self estimated productivity gains from using GitHub Copilot, as reported by the experiment participants in Peng et al. [29]. The treated group (those using Copilot) reported significantly higher productivity gains compared to the control group.

AI-generated code rather than writing it from scratch, which confirms the findings of Bird et al. [7].

5.2.3 Do LLMs really provide productivity gains?

Peng et al. [29] showed strong empirical evidence that AI-assisted programming significantly improves developer productivity. In a controlled experiment, developers using GitHub Copilot completed the task 55.8% faster than those without it, with average completion times of 71 minutes versus 161 minutes for the control group. Beyond overall speed improvements, the results reveal heterogeneous effects across developers. Less experienced programmers, those who spend more hours coding daily, and developers in the 25–44 age range benefited the most from AI assistance. This suggests that LLM-based tools may play a role in lowering barriers to entry and supporting skill development in software engineering. The study also finds that while productivity gains (Figure 12) are substantial, task success rates only improve marginally (around 7 percentage points) and are not statistically significant. This indicates that AI primarily accelerates task completion rather than dramatically increasing correctness.

5.2.4 But at what cost?

LLM-assisted development introduces several notable costs despite its productivity benefits. First, there is a reduction in code understanding and increased cognitive load, as developers shift from writing to reviewing AI-generated code, making debugging harder and reasoning about correctness more challenging [6, 7]. Second, developers face loss of control and over-reliance, often accepting suggestions for speed even when they do not fully understand or verify them, leading to risks such as anchoring bias and misplaced trust [7]. Third, there are reliability and correctness limitations, where generated code may appear plausible but fail in edge cases, introduce subtle bugs, or depend heavily on prompt quality and context [1]. Finally, LLM usage can create workflow inefficiencies in complex scenarios, such as cognitive overload when exploring multiple suggestions or difficulties handling large, multi-file contexts [29]. Overall, these costs highlight that while LLMs accelerate development, they introduce trade-offs in comprehension, control, and reliability, necessitating careful human oversight.

5.3 Debugging

References

[1] A. Agarwal, A. Chan, S. Chandel, J. Jang, S. Miller, R. Z. Moghaddam, Y. Mohylevskyy, N. Sundaresan, and M. Tufano. Copilot evaluation harness: Evaluating llm-guided software programming. *CoRR*, abs/2402.14261, 2024. doi: 10.48550/ARXIV.2402.14261. URL https://doi.org/10.48550/arXiv.2402.14261.

[2] D. Akdur. Analysis of software engineering skills gap in the industry. *ACM Trans. Comput. Educ.*, 23(1), Dec. 2022. doi: 10.1145/3567837. URL https://doi.org/10.1145/3567837.

[3] W. Alhoshan, A. Ferrari, and L. Zhao. Zero-shot learning for requirements classification: An exploratory study. *Information and Software Technology*, 159:107202, 2023. ISSN 0950-5849. doi: https://doi.org/10.1016/j.infsof.2023.107202. URL https://www.sciencedirect.com/science/article/pii/S0950584923000563.

[4] C. Arora, J. Grundy, and M. Abdelrazek. Advancing requirements engineering through generative ai: Assessing the role of llms, 2023. URL https://arxiv.org/abs/2310.13976.

[5] Y. Bang, S. Cahyawijaya, N. Lee, W. Dai, D. Su, B. Wilie, H. Lovenia, Z. Ji, T. Yu, W. Chung, Q. V. Do, Y. Xu, and P. Fung. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity, 2023. URL https://arxiv.org/abs/2302.04023.

[6] S. Barke, M. B. James, and N. Polikarpova. Grounded copilot: How programmers interact with code-generating models. *CoRR*, abs/2206.15000, 2022. doi: 10.48550/ARXIV.2206.15000. URL https://doi.org/10.48550/arXiv.2206.15000.

[7] C. Bird, D. Ford, T. Zimmermann, N. Forsgren, E. Kalliamvakou, T. Lowdermilk, and I. Gazit. Taking flight with copilot. *Commun. ACM*, 66(6):56–62, May 2023. ISSN 0001-0782. doi: 10.1145/3589996. URL https://doi.org/10.1145/3589996.

[8] R. Bommasani, D. A. Hudson, E. Adeli, R. Altman, S. Arora, S. von Arx, M. S. Bernstein, J. Bohg, A. Bosselut, E. Brunskill, E. Brynjolfsson, S. Buch, D. Card, R. Castellon, N. Chatterji, A. Chen, K. Creel, J. Q. Davis, D. Demszky, C. Donahue, M. Doumbouya, E. Durmus, S. Ermon, J. Etchemendy, K. Ethayarajh, L. Fei-Fei, C. Finn, T. Gale, L. Gillespie, K. Goel, N. Goodman, S. Grossman, N. Guha, T. Hashimoto, P. Henderson, J. Hewitt, D. E. Ho, J. Hong,

K. Hsu, J. Huang, T. Icard, S. Jain, D. Jurafsky, P. Kalluri, S. Karamcheti, G. Keeling, F. Khani, O. Khattab, P. W. Koh, M. Krass, R. Krishna, R. Kuditipudi, A. Kumar, F. Ladhak, M. Lee, T. Lee, J. Leskovec, I. Levent, X. L. Li, X. Li, T. Ma, A. Malik, C. D. Manning, S. Mirchandani, E. Mitchell, Z. Munyikwa, S. Nair, A. Narayan, D. Narayanan, B. Newman, A. Nie, J. C. Niebles, H. Nilforoshan, J. Nyarko, G. Ogut, L. Orr, I. Papadimitriou, J. S. Park, C. Piech, E. Portelance, C. Potts, A. Raghunathan, R. Reich, H. Ren, F. Rong, Y. Roohani, C. Ruiz, J. Ryan, C. Ré, D. Sadigh, S. Sagawa, K. Santhanam, A. Shih, K. Srinivasan, A. Tamkin, R. Taori, A. W. Thomas, F. Tramèr, R. E. Wang, W. Wang, B. Wu, J. Wu, Y. Wu, S. M. Xie, M. Yasunaga, J. You, M. Zaharia, M. Zhang, T. Zhang, X. Zhang, Y. Zhang, L. Zheng, K. Zhou, and P. Liang. On the opportunities and risks of foundation models, 2022. URL https://arxiv.org/abs/2108.07258.

[9] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners, 2020. URL https://arxiv.org/abs/2005.14165.

[10] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021. URL https://arxiv.org/abs/2107.03374.

[11] T. Crawford, S. Duong, R. Fueston, A. Lawani, S. Owoade, A. Uzoka, R. M. Parizi, and A. Yazdinejad. Ai in software engineering: A survey on project management applications, 2023. URL https://arxiv.org/abs/2307.15224.

[12] S. Debnath and P. Spoletini. Designing a virtual client for requirements elicitation interviews. In *Requirements Engineering: Foundation for Software Quality: 26th International Working Conference, REFSQ 2020, Pisa, Italy, March 24–27, 2020, Proceedings*, page 160–166, Berlin, Heidelberg, 2020. Springer-Verlag. ISBN 978-3-030-44428-0. doi: 10.1007/978-3-030-44429-7_12. URL https://doi.org/10.1007/978-3-030-44429-7_12.

[13] K. Ferdowsifard, S. Barke, H. Peleg, S. Lerner, and N. Polikarpova. Loopy: interactive program synthesis with control structures. *Proc. ACM Program. Lang.*, 5(OOPSLA), Oct. 2021. doi: 10.1145/3485530. URL https://doi.org/10.1145/3485530.

[14] GitHub. Github, 2026. URL https://www.github.com. Accessed: 2026-04-10.

[15] Google DeepMind. Competitive programming with alphacode, 2022. URL https://alphacode.deepmind.com/. Accessed: 2026-04-10.

[16] Y. Gu, H. You, J. Cao, M. Yu, H. Fan, and S. Qian. Large language models for constructing and optimizing machine learning workflows: A survey. *ACM Trans. Softw. Eng. Methodol.*, Oct. 2025. ISSN 1049-331X. doi: 10.1145/3773084. URL https://doi.org/10.1145/3773084. Just Accepted.

[17] B. Görer and F. B. Aydemir. Generating requirements elicitation interview scripts with large language models. In *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*, pages 44–51, 2023. doi: 10.1109/REW57809.2023.00015.

[18] B. Görer and F. Başak. Generating requirements elicitation interview scripts with large language models—supplementary material, June 2023. URL https://doi.org/10.5281/zenodo.8049207.

[19] D. Kaur, S. Uslu, and A. Durresi. Requirements for trustworthy artificial intelligence - A review. In L. Barolli, K. F. Li, T. Enokido, and M. Takizawa, editors, *Advances in Networked-Based Information Systems - The 23rd International Conference on Network-Based Information Systems, NBiS 2020, Victoria, BC, Canada, 31 August - 2 September 2020*, Advances in Intelligent Systems and Computing, pages 105–115. Springer, 2020. doi: 10.1007/978-3-030-57811-4_11. URL https://doi.org/10.1007/978-3-030-57811-4_11.

[20] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman. Studying the effect of ai code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394215. doi: 10.1145/3544548.3580919. URL https://doi.org/10.1145/3544548.3580919.

[21] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago, T. Hubert, P. Choy, C. de Masson d'Autume, I. Babuschkin, X. Chen, P.-S. Huang, J. Welbl, S. Gowal, A. Cherepanov, J. Molloy, D. J. Mankowitz, E. Sutherland Robson, P. Kohli, N. de Freitas, K. Kavukcuoglu, and O. Vinyals. Competition-level code generation with alphacode. *Science*, 378(6624): 1092–1097, Dec. 2022. ISSN 1095-9203. doi: 10.1126/science.abq1158. URL http://dx.doi.org/10.1126/science.abq1158.

[22] T. Mao, D. Zhao, H. Tang, X. Wang, and H. Zhang. A large-scale empirical study of ai-generated code in real-world repositories, 2026. URL https://arxiv.org/abs/2603.27130.

[23] McKinsey & Company. The economic potential of generative ai, 2023. URL https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier. Industry report.

[24] A. Miltner, S. Gulwani, V. Le, A. Leung, A. Radhakrishna, G. Soares, A. Tiwari, and A. Udupa. On the fly synthesis of edit suggestions. *Proc. ACM Program. Lang.*, 3(OOPSLA),

Oct. 2019. doi: 10.1145/3360569. URL https://doi.org/10.1145/3360569.

[25] A. H. Mohammadkhani, N. S. Bommi, M. Daboussi, O. Sabnis, C. Tantithamthavorn, and H. Hemmati. A systematic literature review of explainable ai for software engineering, 2023. URL https://arxiv.org/abs/2302.06065.

[26] A. Nguyen-Duc, B. Cabrero-Daniel, A. Przybylek, C. Arora, D. Khanna, T. Herda, U. Rafiq, J. Melegati, E. Guerra, K.-K. Kemell, M. Saari, Z. Zhang, H. Le, T. Quan, and P. Abrahamsson. Generative artificial intelligence for software engineering—a research agenda. *Software: Practice and Experience*, 55(11):1806–1843, 2025. doi: https://doi.org/10.1002/spe.70005. URL https://onlinelibrary.wiley.com/doi/abs/10.1002/spe.70005.

[27] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong. Codegen: An open large language model for code with multi-turn program synthesis, 2023. URL https://arxiv.org/abs/2203.13474.

[28] OpenAI. Openai codex models, 2026. URL https://developers.openai.com/codex/models. Accessed: 2026-04-10.

[29] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer. The impact of AI on developer productivity: Evidence from github copilot. *CoRR*, abs/2302.06590, 2023. doi: 10.48550/ARXIV.2302.06590. URL https://doi.org/10.48550/arXiv.2302.06590.

[30] Z. Rasheed, M. A. Sami, M. Waseem, K.-K. Kemell, X. Wang, A. Nguyen, K. Systä, and P. Abrahamsson. Ai-powered code review with llms: Early results, 2025. URL https://arxiv.org/abs/2404.18496.

[31] V. Raychev, M. Vechev, and E. Yahav. Code completion with statistical language models. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '14, page 419–428, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450327848. doi: 10.1145/2594291.2594321. URL https://doi.org/10.1145/2594291.2594321.

[32] K. Ronanki, C. Berger, and J. Horkoff. Investigating chatgpt's potential to assist in requirements elicitation processes. In *49th Euromicro Conference on Software Engineering and Advanced Applications, SEAA 2023, Durres, Albania, September 6-8, 2023*, pages 354–361. IEEE, 2023. doi: 10.1109/SEAA60479.2023.00061. URL https://doi.org/10.1109/SEAA60479.2023.00061.

[33] P. Vaithilingam, T. Zhang, and E. L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI EA '22, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450391566. doi: 10.1145/3491101.3519665. URL https://doi.org/10.1145/3491101.3519665.

[34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, 2023. URL https://arxiv.org/abs/1706.03762.

[35] E. Wastl. Advent of code, 2023. URL https://adventofcode.com/.

[36] F. F. Xu, Z. Jiang, P. Yin, B. Vasilescu, and G. Neubig. Incorporating external knowledge through pre-training for natural language to code generation, 2020. URL https://arxiv.org/abs/2004.09015.